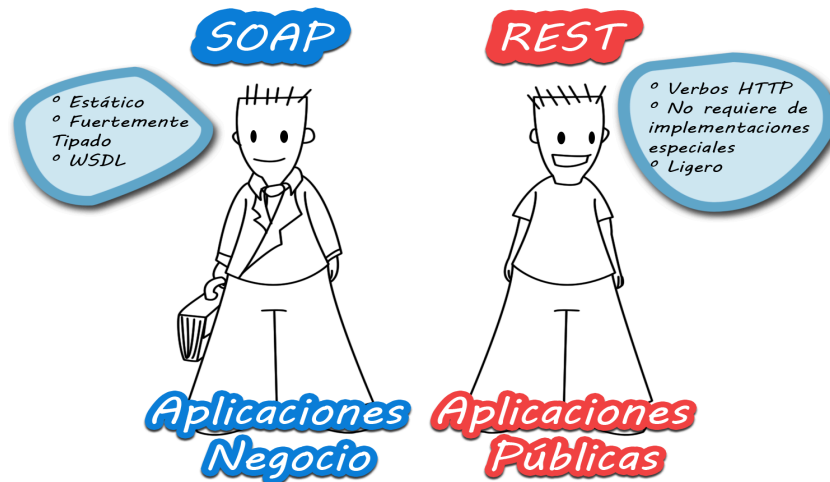


Actividad 3.1: Comparativa entre Servicios Web SOAP y REST



Nombre de la asignatura: Programación de Servicios Web

Actividad 3.1 Comparativa entre Servicios Web SOAP y REST

Nombre completo del estudiante:

- Hernández García, Héctor Gabriel
- Mexicano Romero, Gerardo Misael
- Pardo Correa, Gonzalo
- Ramírez Torres, Jesús Adrián

Objetivo

Analizar las características, funcionamiento y aplicaciones de los servicios web SOAP y REST, identificando sus semejanzas, diferencias, ventajas y desventajas. Al finalizar la actividad, el estudiante comprenderá los escenarios en los que resulta conveniente utilizar cada arquitectura mediante la elaboración de una infografía comparativa con ejemplos prácticos.

Enlaces del proyecto

Repositorio GitHub: Enlace directo al código fuente.

Video Demostrativo (Máx. 5 min): <https://youtu.be/ecGohnWpUaY>

Repositorio de GitHub: <https://github.com/Andyor-prog/Farmacia.git>

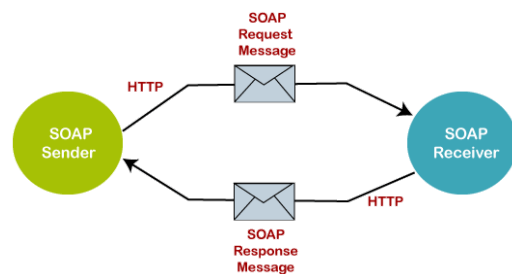
SOAP vs REST Dos formas de comunicar aplicaciones y compartir información a través de servicios web

SOAP

Simple Object Access Protocol es un protocolo de comunicación que define una estructura formal para intercambiar mensajes XML entre sistemas. Puede utilizar distintos protocolos de transporte y suele describir operaciones mediante WSDL.

Se distingue por

- Mensajes XML con sobre, encabezado y cuerpo.
- Contrato WSDL frecuente.
- Estándares complementarios, como WS-Security.
- Reglas estrictas de mensajería.

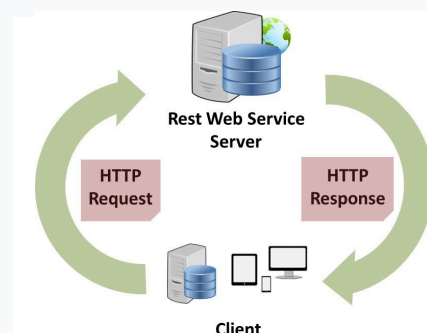


REST

Representational State Transfer es un estilo arquitectónico que organiza una API alrededor de recursos identificados por URL y utiliza operaciones HTTP para consultar o modificar su representación.

Se distingue por

- Recursos y URI.
- Métodos HTTP: GET, POST, PUT, PATCH y DELETE.
- JSON es habitual; también admite XML y otros formatos.
- Comunicación sin estado como principio REST.



¿Cómo funciona cada uno?

SOAP

Cliente → mensaje XML (Envelope) → servicio procesa operación → respuesta XML o Fault

REST

Cliente → método HTTP + URI → servidor opera sobre recurso → respuesta HTTP (frecuentemente JSON)

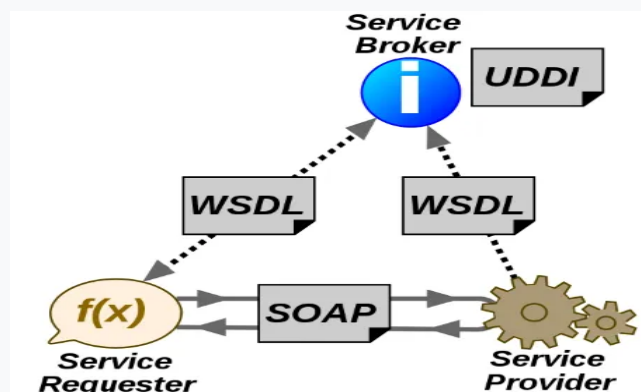
Semejanzas

Ambos permiten integrar sistemas desarrollados con diferentes tecnologías, intercambian datos mediante solicitudes y respuestas, pueden protegerse con HTTPS y se utilizan para conectar aplicaciones, servicios y plataformas.

Componentes y términos clave:

SOAP: Envelope, Header, Body, Fault, WSDL y, cuando aplica, WS-* (estándares complementarios).

REST: recursos, URI, métodos HTTP, representación, códigos de estado, encabezados y caché.



Comparación técnica

La elección depende de los requisitos del sistema: contratación, seguridad, rendimiento, integración y facilidad de evolución.

Aspecto	SOAP	REST
Naturaleza	Protocolo con reglas de mensaje definidas.	Estilo arquitectónico, no protocolo.
Transporte	Puede usar HTTP/HTTPS y otros transportes compatibles.	Principalmente HTTP/HTTPS.
Formato	XML obligatorio para el mensaje SOAP.	Flexible; JSON frecuente, también XML, texto y otros.
Contrato	WSDL describe operaciones y mensajes.	OpenAPI puede documentar la API; no es requisito REST.
Flexibilidad	Más rígido y estandarizado.	Más flexible en recursos y representaciones
Rendimiento	XML y envoltura pueden añadir sobrecarga.	Puede ser más ligero; depende del diseño y payload
Seguridad	WS-Security y TLS/HTTPS, según implementación.	TLS/HTTPS, OAuth 2.0, tokens y controles de API, según implementación.
Implementación	A menudo requiere herramientas y configuración especializadas	Suele ser sencillo con frameworks web comunes.
Escalabilidad	Puede escalar; depende de diseño, estado y despliegue.	Stateless y caché pueden facilitar el escalamiento horizontal.
Compatibilidad	Muy utilizado en integración empresarial y sistemas heredados.	Amplio soporte en web, móviles y servicios cloud.
Aplicaciones	Operaciones empresariales con contratos y políticas formales.	APIs web/móvil, microservicios y acceso a recursos.

Ventajas y desventajas

SOAP · Ventajas

- Mensajería estandarizada y contrato explícito.
- WS-Security y extensiones para necesidades empresariales.
- Manejo formal de fallos.

REST · Ventajas

- Sencillo y ampliamente soportado.
- Respuestas JSON compactas en muchos casos.
- Aprovecha semántica HTTP y caché.

SOAP · Desventajas

- Mensajes más verbosos.
- Mayor complejidad de implementación y mantenimiento.
- Menos práctico para clientes web ligeros.

REST · Desventajas

- La seguridad y el contrato deben diseñarse/documentarse.
- Diseños inconsistentes dificultan interoperabilidad.
- No define por sí solo transacciones complejas.

REST no garantiza automáticamente mejor rendimiento o seguridad; estos dependen de la implementación, el volumen de datos, autenticación, autorización, validación y despliegue.

Seis ejemplos de uso real

Los ejemplos muestran productos o interfaces conocidos. Algunos proveedores mantienen varias versiones o estilos de API; se especifica el servicio concreto para evitar confundirlo con otras interfaces.



SOAP · Salesforce SOAP API Funcionamiento:

Integra aplicaciones externas con el CRM mediante operaciones SOAP descritas por WSDL. Problema: Resuelve sincronización e integración de procesos de ventas y atención al cliente. Información: Cuentas, contactos, oportunidades y registros de CRM.

SOAP · Microsoft Dynamics CRM SOAP / Organization Service (versiones clásicas) Funcionamiento:

Permite que aplicaciones cliente ejecuten operaciones de negocio contra Dynamics mediante mensajes SOAP. Problema: Conecta sistemas empresariales y automatiza operaciones de CRM, especialmente en entornos existentes. Información: Cuentas, contactos, actividades, entidades y operaciones.

SOAP · eBay Trading API (interfaz SOAP) Funcionamiento:

Interfaz de comercio que históricamente permite gestionar publicaciones y operaciones de vendedor con solicitudes SOAP. Problema: Automatiza la administración de catálogos y ventas en eBay. Información: Artículos, inventario, pedidos, ventas y datos de publicación.

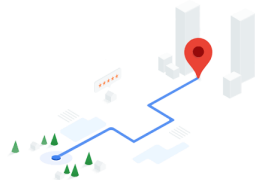
REST · GitHub REST API Funcionamiento:

Permite consultar y administrar repositorios y recursos mediante endpoints HTTP. Problema: Automatiza flujos de desarrollo, seguimiento de incidencias e integración continua. Información: Repositorios, commits, issues, pull requests y usuarios.



REST · Google Maps Platform Web Services Funcionamiento:

APIs HTTP permiten solicitar geocodificación, rutas, distancias y otros datos geográficos. Problema: Integra mapas, localización y cálculo de trayectos en aplicaciones. Información: Direcciones, coordenadas, rutas, distancias y lugares.



REST · Spotify Web API Funcionamiento:

Permite a aplicaciones autorizadas consultar catálogo musical y recursos de cuenta mediante HTTP. Problema: Integra datos musicales y funciones compatibles en aplicaciones externas. Información: Artistas, álbumes, pistas, playlists y datos autorizados.

Aplicación al Proyecto Farmacia

1. Diagnóstico del Estado Actual del Sistema

Actualmente, el proyecto está desarrollado sobre el framework Laravel (PHP) bajo un patrón MVC monolítico. En el archivo routes/web.php se gestionan las operaciones CRUD del catálogo de productos mediante vistas tradicionales (Blade).

En routes/api.php se incluye únicamente una ruta autenticada mediante Laravel Sanctum. Por lo tanto, aunque el proyecto cuenta con infraestructura base para APIs, no dispone aún de un servicio RESTful completo para la gestión de productos.

2. Propuesta de Diseño API RESTful (Catálogo de Productos)

Para permitir la integración con aplicaciones móviles, puntos de venta (POS) y servicios externos de inventario, se propone la siguiente arquitectura de endpoints bajo principios REST:

Método HTTP	Endpoint / Ruta	Descripción de la Operación	Código de Estado / Respuesta
GET	/api/productos	Obtener el catálogo completo de productos	200 OK (Colección JSON)
GET	/api/productos/{id}	Consultar los detalles de un producto específico	200 OK / 404 Not Found
POST	/api/productos	Registrar un nuevo producto en el inventario	201 Created (Objeto JSON)
PUT / PATCH	/api/productos/{id}	Actualizar datos totales o parciales del producto	200 OK / 422 Unprocessable
DELETE	/api/productos/{id}	Dar de baja un producto del sistema	204 No Content

Nota de arquitectura: Esta propuesta define el contrato de la API bajo comunicación sin estado (stateless) utilizando JSON como formato estándar de intercambio.

[Cliente]

[API Laravel]

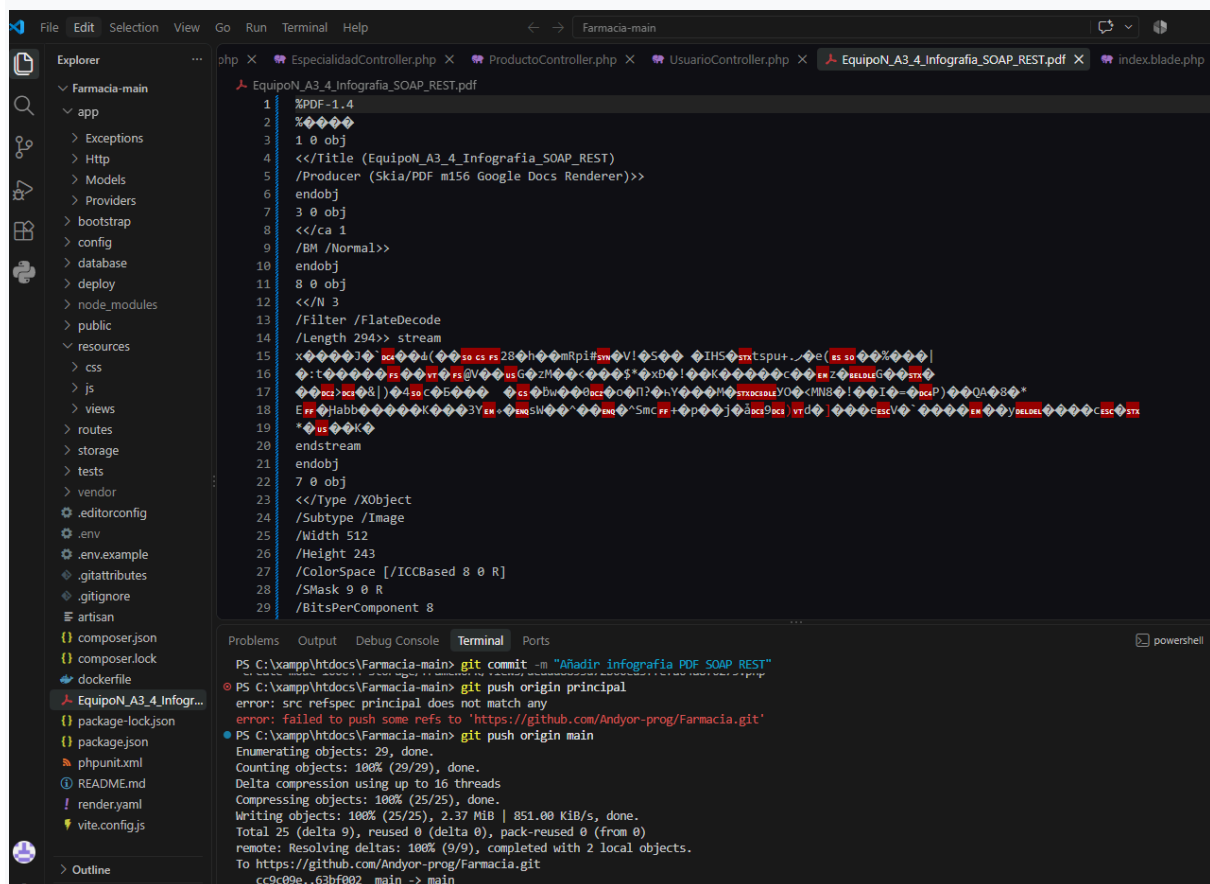
[Base de Datos]

(Navegador / App) — Solicita HTTP (JSON) —> (Rutas / Controllers) — Consulta SQL —> (MySQL / PostgreSQL)

← Respuesta + Status —

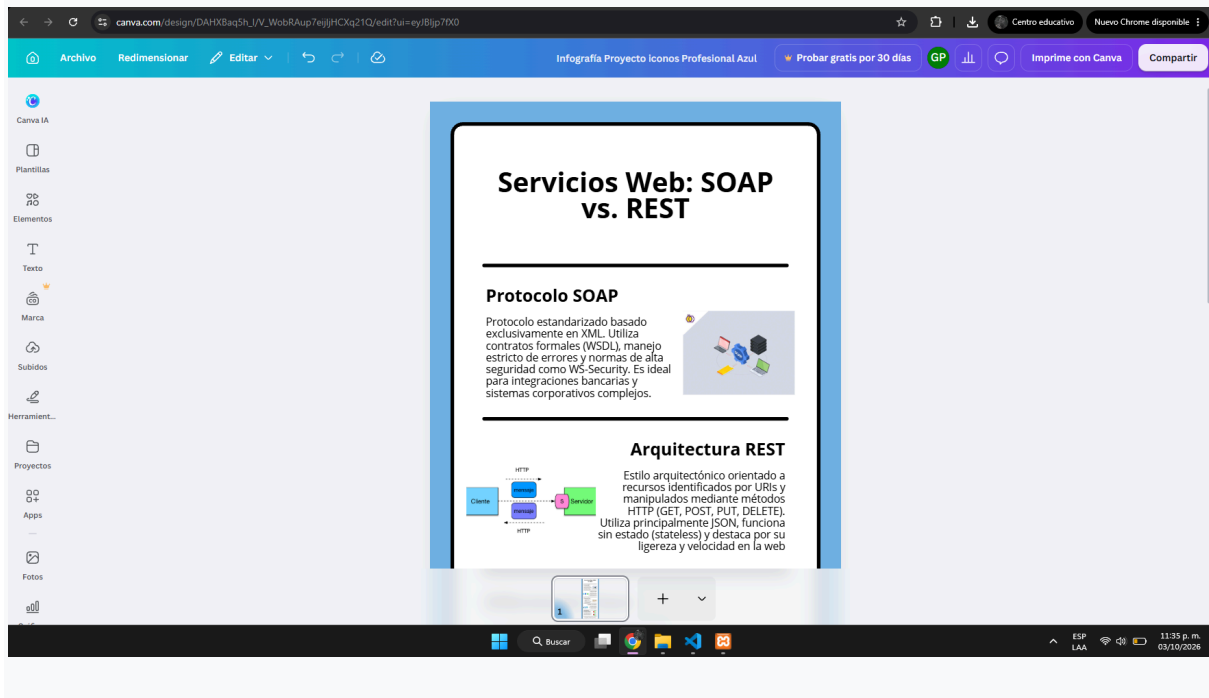
← Devuelve Modelo

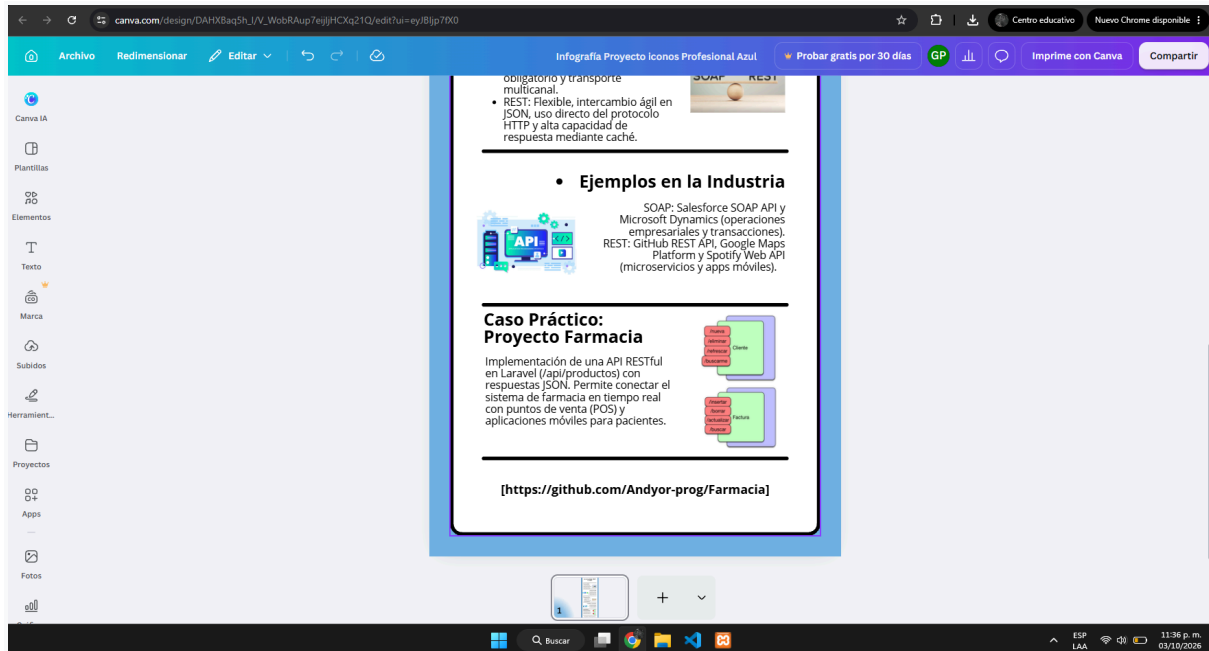
Evidencias



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'Farmacia-main' with various folders and files. The main editor area displays a PDF file named 'EquipoN_A3_4_Infografia_SOAP_REST.pdf'. The terminal window shows the following commands and output:

```
PS C:\xampp\htdocs\Farmacia-main> git commit -m "Añadir infografia PDF SOAP REST"
PS C:\xampp\htdocs\Farmacia-main> git push origin principal
error: src refspec principal does not match any
error: failed to push some refs to 'https://github.com/Andyor-prog/Farmacia.git'
PS C:\xampp\htdocs\Farmacia-main> git push origin main
Enumerating objects: 29, done.
Counting objects: 100% (29/29), done.
Delta compression using up to 16 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (25/25), 2.37 MiB | 851.00 KiB/s, done.
Total 25 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (9/9), completed with 2 local objects.
To https://github.com/Andyor-prog/Farmacia.git
cc9c09e..63bf002 main -> main
```





Conclusión del Equipo

Tanto SOAP como REST permiten la integración de sistemas distribuidos, pero responden a necesidades distintas:

- SOAP ofrece un protocolo estricto basado en contratos XML (WSDL) y estándares de seguridad robustos (WS-Security), ideal para entornos financieros o gubernamentales.
- REST es la solución idónea para el Proyecto Farmacia, ya que proporciona ligereza, consumo reducido de ancho de banda mediante JSON, facilidad de integración con clientes web/móviles y alta escalabilidad.

La implementación de este diseño RESTful garantizará que el módulo de farmacia pueda conectarse de forma ágil con subsistemas de facturación, aplicaciones móviles para pacientes y control de stock en tiempo real.

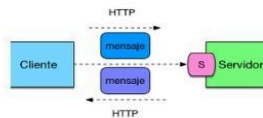
Servicios Web: SOAP vs. REST

Protocolo SOAP

Protocolo estandarizado basado exclusivamente en XML. Utiliza contratos formales (WSDL), manejo estricto de errores y normas de alta seguridad como WS-Security. Es ideal para integraciones bancarias y sistemas corporativos complejos.



Arquitectura REST



Estilo arquitectónico orientado a recursos identificados por URIs y manipulados mediante métodos HTTP (GET, POST, PUT, DELETE). Utiliza principalmente JSON, funciona sin estado (stateless) y destaca por su ligereza y velocidad en la web.

Diferencias Clave

- SOAP: Rígido, mensajes pesados en XML, contrato WSDL obligatorio y transporte multicanal.
- REST: Flexible, intercambio ágil en JSON, uso directo del protocolo HTTP y alta capacidad de respuesta mediante caché.



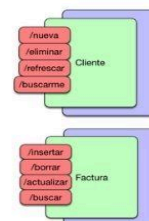
Ejemplos en la Industria



SOAP: Salesforce SOAP API y Microsoft Dynamics (operaciones empresariales y transacciones).
REST: GitHub REST API, Google Maps Platform y Spotify Web API (microservicios y apps móviles).

Caso Práctico: Proyecto Farmacia

Implementación de una API RESTful en Laravel (/api/productos) con respuestas JSON. Permite conectar el sistema de farmacia en tiempo real con puntos de venta (POS) y aplicaciones móviles para pacientes.



[<https://github.com/Andyor-prog/Farmacia>]